

cpu_bound 函数全面解析（基于 NiceGUI）

`cpu_bound` 是 NiceGUI 框架中 `run` 模块下的核心函数，专门用于**异步执行 CPU 密集型任务**，解决 UI 主线程阻塞、界面无响应的问题。以下从核心特性、工作原理、使用规则、示例拆解等维度详细说明：

一、核心定位与解决的问题

CPU 密集型任务（如复杂计算、大数据处理、循环运算）会长时间占用 CPU，若直接在 NiceGUI 的事件循环（UI 主线程）中执行，会导致：

- UI 界面卡死（按钮点击、输入等操作无响应）；
- 事件循环阻塞，其他异步任务（如网络请求、定时器）无法执行。

`cpu_bound` 的核心作用：**将 CPU 密集型任务分发到独立的子进程中执行**，主线程（UI）可继续响应交互，任务完成后通过异步 Future 返回结果。

二、工作原理

1. 进程隔离：

`cpu_bound` 基于 Python 的多进程（而非多线程）实现 —— 因为 Python 的 GIL（全局解释器锁）会导致多线程无法真正并行执行 CPU 密集型任务，而多进程可绕过 GIL，充分利用多核 CPU。

2. 序列化传输：

子进程与主进程之间的参数 / 结果传递依赖 `pickle` 序列化：

- 主进程将待执行的函数、传入的参数序列化后，发送给子进程；
- 子进程执行函数，将结果序列化后返回主进程；
- 主进程反序列化结果，通过 Future 对象让异步函数（如示例中的 `handle_click`）获取结果。

3. 异步封装：

函数返回一个可 `await` 的 Future 对象，符合 NiceGUI 的异步事件处理逻辑，无需手动管理进程的启动 / 等待。

三、使用规则与最佳实践

官方明确的约束和建议是使用该函数的关键：

1. 函数类型要求：

- 优先使用「无状态的自由函数（顶层函数）」或「静态方法」，避免使用类实例方法、闭包；
- 原因：类实例 / 闭包包含上下文引用，`pickle` 序列化时易出错（如无法序列化 UI 组件、类实例）。

2. 参数 / 返回值要求：

- 参数和返回值必须是 `pickle` 可序列化的类型（如 `int/float/str/list/dict` 等基础类型）；
- 禁止传递 UI 组件（如 `ui.button`、`ui.label`）、文件句柄、网络连接等不可序列化对象。

3. 数据传递方式：

- 所有需要处理的数据必须通过参数显式传入，禁止依赖全局变量、类属性；
- 结果必须通过函数返回值传递，禁止直接修改全局 / 类属性（子进程无法修改主进程的变量）。

四、示例代码逐行拆解

以官方示例为例，理解每一步的设计逻辑：

```
import time
from nicegui import run, ui

# 1. 定义CPU密集型任务：自由函数+纯基础类型参数/返回值
def compute_sum(a: float, b: float) -> float:
    time.sleep(1) # 模拟耗时计算（替代真实的CPU密集逻辑）
    return a + b # 结果通过返回值传递，不依赖任何外部状态

# 2. 异步事件处理函数：UI交互的入口
async def handle_click():
    # 3. 调用cpu_bound：传入函数+参数，await获取结果
    result = await run.cpu_bound(compute_sum, 1, 2)
    # 4. 结果回显到UI：主进程处理UI更新
    ui.notify(f'Sum is {result}')

# 5. UI组件绑定异步处理函数
ui.button('Compute', on_click=handle_click)

ui.run()
```

- `compute_sum` 是纯函数（无外部依赖），符合序列化要求；
- `handle_click` 是异步函数，保证 UI 线程不阻塞；
- `run.cpu_bound(compute_sum, 1, 2)` 中，`1` 和 `2` 是基础类型，可被 pickle 序列化；
- 结果通过返回值获取后，在主进程中更新 UI（子进程无法直接操作 UI）。

五、常见错误与避坑

1. 错误示例 1：依赖全局变量（子进程无法读取主进程全局变量的最新值）

```
# 错误写法
total = 0
def compute_sum(a, b):
    global total
    total = a + b # 子进程修改的是自己的全局变量，主进程无法获取
    return None

async def handle_click():
    await run.cpu_bound(compute_sum, 1, 2)
    ui.notify(f'Sum is {total}') # 输出仍为0
```

2. 错误示例 2：传递 UI 组件（pickle 序列化失败）

错误写法

```
def compute_sum(a, b, label):  
    result = a + b  
    label.set_text(result) # 试图修改UI组件，直接报错  
    return result  
  
async def handle_click():  
    label = ui.label()  
    await run.cpu_bound(compute_sum, 1, 2, label) # 序列化label失败
```